Assignment No: 15

Title: Develop a REST API using Node.js, Express, and database connectivity.

Aim: To develop a RESTful API using Node.js and Express that is backed by a MySQL database, exposing the full set of CRUD endpoints for a manuscript resource with correct HTTP verbs, status codes, validation, pagination and error handling.

Objectives:

1. To understand the principles of REST and resource-oriented URL design.
2. To map the CRUD operations onto the HTTP verbs GET, POST, PUT, PATCH and DELETE.
3. To connect Express to MySQL using a connection pool and the promise API.
4. To implement the endpoints with parameterised queries that prevent SQL injection.
5. To validate request bodies and return HTTP 400 with a useful message.
6. To return the correct status code for every outcome: 200, 201, 204, 400, 404, 409, 500.
7. To implement pagination and filtering through query-string parameters.
8. To enable CORS so that the React client can consume the API from another origin.
9. To keep credentials out of source control using environment variables.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (v18 or later) and npm
- Express 5.x, mysql2, cors, dotenv
- MySQL / MariaDB server (XAMPP)
- Postman or curl (for testing the endpoints)

Theory:

1. REST

REST (Representational State Transfer) is an architectural style for web services. A REST API exposes resources at URLs, acts on them with the standard HTTP verbs, and transfers their representation - normally JSON. It is stateless: every request carries everything the server needs, so no client context is held between calls.

- Resources are named with nouns, not verbs: /manuscripts, not /getManuscripts.
- The verb carries the intent, so one URL serves several operations.

- A collection URL is plural; an item URL appends the identifier.
- Responses are uniform and self-describing, with a meaningful status code.

2. Endpoint Design

Verb and path	Operation	Success code
GET /api/manuscripts	List, with filtering and pagination	200 OK
GET /api/manuscripts/:id	Read one	200 OK
POST /api/manuscripts	Create	201 Created
PUT /api/manuscripts/:id	Replace the whole record	200 OK
PATCH /api/manuscripts/:id	Update some fields	200 OK
DELETE /api/manuscripts/:id	Delete	204 No Content

3. HTTP Status Codes in a REST API

Code	Returned when
200 OK	A read, update or action succeeded and a body is returned
201 Created	A resource was created; the Location header names it
204 No Content	The action succeeded and there is nothing to return
400 Bad Request	Validation of the body or parameters failed
404 Not Found	The identified resource does not exist
409 Conflict	The request clashes with the current state, e.g. a duplicate key
500 Internal Server Error	An unexpected failure on the server

4. Database Connectivity with mysql2

The mysql2 driver offers a promise-based API that works with async/await, and a connection pool that reuses a fixed set of connections instead of opening one per request. Queries use ? placeholders and an array of values, so the driver escapes every value and SQL injection is not possible.

Pool and query:

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10
});

const [rows] = await pool.query(
  'SELECT * FROM manuscripts WHERE script = ?', [script]
);
```

5. CORS

A browser blocks a request from one origin to another unless the server opts in with Cross-Origin Resource Sharing headers. The cors middleware adds them. Without it the React client at http://localhost:5173 could not call an API at http://localhost:3000.

6. Environment Variables

Database credentials must not be written into the source. dotenv reads a .env file into process.env at startup, and .env is listed in .gitignore while a .env.example with placeholder values is committed instead.

7. Layering and Error Handling

Routes are kept thin: a router module declares the endpoints, each handler validates its input and calls the database, and a single error-handling middleware converts any thrown error into a JSON response. Wrapping each async handler so that a rejected promise reaches next() avoids an unhandled rejection crashing the process.

Course Project Implementation:

Course-Project Mapping: Not available in the course project. LipiSutra exposes no API of its own and runs no server-side database code; src/services/firebase.js talks to Firestore straight from the browser using the Firebase Web SDK, and src/services/gemini.js calls Google's API with a key that is bundled into the client. That key exposure is exactly the problem a REST API layer solves, so the API below is written as the backend LipiSutra should adopt: the React client would call /api/manuscripts instead of Google directly, and the keys would stay on the server. There is no existing project file to submit for this assignment.

Requirement	In project?	Nearest equivalent
REST endpoints	No	- (the client calls third-party APIs directly)
Server-side DB connectivity	No	src/services/firebase.js, which runs in the browser
Verb-based CRUD	Conceptually	setDoc / getDoc / updateDoc in firebase.js
Secrets kept server-side	No	VITE_GEMINI_KEY is bundled into the client build

Program:

File 1: package.json

```
{
  "name": "lipisutra-api",
  "version": "1.0.0",
  "type": "commonjs",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev":   "node --watch server.js"
  },
  "dependencies": {
    "cors":    "^2.8.5",
    "dotenv":  "^17.2.3",
    "express": "^5.1.0",
    "mysql2":  "^3.15.3"
```

```
  }
}
```

File 2: .env (listed in .gitignore; .env.example is committed instead)

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASS=
DB_NAME=lipisutra
CLIENT_ORIGIN=http://localhost:5173
```

File 3: db.js - the connection pool

```
require("dotenv").config();
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host:     process.env.DB_HOST,
  user:     process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  charset:  "utf8mb4",          // Devanagari / Modi transcripts
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

// fail fast at start-up rather than on the first request
(async () => {
  try {
    const conn = await pool.getConnection();
    await conn.ping();
    conn.release();
    console.log("MySQL pool ready.");
  } catch (err) {
    console.error("MySQL connection failed:", err.message);
    process.exit(1);
  }
})();

module.exports = pool;
```

File 4: routes/manuscripts.js - the REST endpoints

```
const express = require("express");
const pool    = require("../db");

const router = express.Router();

const SCRIPTS  = ["Modi", "Devanagari", "Brahmi", "Sharada"];
const STATUSES = ["pending", "verified"];

/* wrap an async handler so a rejected promise reaches the error middleware */
const wrap = fn => (req, res, next) => Promise.resolve(fn(req, res,
next)).catch(next);

/* ---------- validation helper ---------- */
function validate(body, { partial = false } = {}) {
  const errors = [];
  const has = k => body[k] !== undefined && body[k] !== null && body[k] !== "";

  if (!partial || has("accession")) {
    if (!/^LS-\d{4}$/.test(body.accession || ""))
      errors.push("accession must match LS-0000");
  }
```

```
  if (!partial || has("title")) {
    if (!body.title || body.title.trim().length < 5)
      errors.push("title must be at least 5 characters");
  }
  if (!partial || has("script")) {
    if (!SCRIPTS.includes(body.script))
      errors.push(`script must be one of ${SCRIPTS.join(", ")}`);
  }
  if (!partial || has("era_year")) {
    const y = Number(body.era_year);
    if (!Number.isInteger(y) || y < -500 || y > 2026)
      errors.push("era_year must be a whole number between -500 and 2026");
  }
  if (has("confidence")) {
    const c = Number(body.confidence);
    if (Number.isNaN(c) || c < 0 || c > 100)
      errors.push("confidence must be between 0 and 100");
  }
  if (has("status") && !STATUSES.includes(body.status)) {
    errors.push(`status must be one of ${STATUSES.join(", ")}`);
  }
  return errors;
}

/* ======================================================================
   GET /api/manuscripts
   Filtering  : ?script=Modi&status=verified&q=grant
   Pagination : ?page=1&limit=10
   ====================================================================== */
router.get("/", wrap(async (req, res) => {
  const page  = Math.max(1, parseInt(req.query.page)  || 1);
  const limit = Math.min(50, Math.max(1, parseInt(req.query.limit) || 10));
  const offset = (page - 1) * limit;

  const where  = [];
  const params = [];

  if (req.query.script) { where.push("script = ?"); params.push(req.query.script); }
  if (req.query.status) { where.push("status = ?"); params.push(req.query.status); }
  if (req.query.q) {
    where.push("(title LIKE ? OR accession LIKE ?)");
    params.push(`%${req.query.q}%`, `%${req.query.q}%`);
  }

  const clause = where.length ? "WHERE " + where.join(" AND ") : "";

  const [[{ total }]] = await pool.query(
    `SELECT COUNT(*) AS total FROM manuscripts ${clause}`, params
  );

  const [rows] = await pool.query(
    `SELECT * FROM manuscripts ${clause}
     ORDER BY created_at DESC
     LIMIT ? OFFSET ?`,
    [...params, limit, offset]
  );

  res.status(200).json({
    page, limit, total,
    pages: Math.ceil(total / limit),
    data: rows
  });
}));

/* ======================================================================
   GET /api/manuscripts/:id
   ====================================================================== */
router.get("/:id", wrap(async (req, res) => {
  const id = Number(req.params.id);
```

```
  if (!Number.isInteger(id) || id < 1) {
    return res.status(400).json({ error: "id must be a positive integer" });
  }

  const [rows] = await pool.query("SELECT * FROM manuscripts WHERE id = ?", [id]);

  if (rows.length === 0) {
    return res.status(404).json({ error: `No manuscript with id ${id}` });
  }
  res.status(200).json(rows[0]);
}));

/* ======================================================================
   POST /api/manuscripts        -> 201 Created
   ====================================================================== */
router.post("/", wrap(async (req, res) => {
  const errors = validate(req.body);
  if (errors.length) return res.status(400).json({ errors });

  const { accession, title, script, era_year,
          confidence = 0, status = "pending", transcript = null } = req.body;

  try {
    const [result] = await pool.query(
      `INSERT INTO manuscripts
         (accession, title, script, era_year, confidence, status, transcript)
       VALUES (?, ?, ?, ?, ?, ?, ?)`,
      [accession, title.trim(), script, era_year, confidence, status, transcript]
    );

    const [rows] = await pool.query(
      "SELECT * FROM manuscripts WHERE id = ?", [result.insertId]
    );

    res.status(201)
       .location(`/api/manuscripts/${result.insertId}`)
       .json(rows[0]);

  } catch (err) {
    if (err.code === "ER_DUP_ENTRY") {
      return res.status(409).json({ error: `accession ${accession} already exists` });
    }
    throw err;                       // anything else goes to the error middleware
  }
}));

/* ======================================================================
   PUT /api/manuscripts/:id      -> full replacement
   ====================================================================== */
router.put("/:id", wrap(async (req, res) => {
  const id = Number(req.params.id);
  const errors = validate(req.body);
  if (errors.length) return res.status(400).json({ errors });

  const { accession, title, script, era_year,
          confidence = 0, status = "pending", transcript = null } = req.body;

  const [result] = await pool.query(
    `UPDATE manuscripts
        SET accession = ?, title = ?, script = ?, era_year = ?,
            confidence = ?, status = ?, transcript = ?
      WHERE id = ?`,
    [accession, title.trim(), script, era_year, confidence, status, transcript, id]
  );

  if (result.affectedRows === 0) {
    return res.status(404).json({ error: `No manuscript with id ${id}` });
  }
```

```
  const [rows] = await pool.query("SELECT * FROM manuscripts WHERE id = ?", [id]);
  res.status(200).json(rows[0]);
}));

/* ======================================================================
   PATCH /api/manuscripts/:id    -> partial update
   ====================================================================== */
router.patch("/:id", wrap(async (req, res) => {
  const id = Number(req.params.id);

  const errors = validate(req.body, { partial: true });
  if (errors.length) return res.status(400).json({ errors });

  const allowed = ["accession", "title", "script", "era_year",
                   "confidence", "status", "transcript"];
  const sets   = [];
  const params = [];

  allowed.forEach(field => {
    if (req.body[field] !== undefined) {
      sets.push(`${field} = ?`);         // field name comes from a fixed allow-list
      params.push(req.body[field]);
    }
  });

  if (sets.length === 0) {
    return res.status(400).json({ error: "Supply at least one field to update" });
  }

  params.push(id);
  const [result] = await pool.query(
    `UPDATE manuscripts SET ${sets.join(", ")} WHERE id = ?`, params
  );

  if (result.affectedRows === 0) {
    return res.status(404).json({ error: `No manuscript with id ${id}` });
  }

  const [rows] = await pool.query("SELECT * FROM manuscripts WHERE id = ?", [id]);
  res.status(200).json(rows[0]);
}));

/* ======================================================================
   DELETE /api/manuscripts/:id   -> 204 No Content
   ====================================================================== */
router.delete("/:id", wrap(async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1) {
    return res.status(400).json({ error: "id must be a positive integer" });
  }

  const [result] = await pool.query("DELETE FROM manuscripts WHERE id = ?", [id]);

  if (result.affectedRows === 0) {
    return res.status(404).json({ error: `No manuscript with id ${id}` });
  }
  res.status(204).send();              // success, nothing to return
}));

module.exports = router;
```

File 5: server.js - wiring the API together

```
require("dotenv").config();
```

```
const express = require("express");
const cors    = require("cors");

const manuscriptRoutes = require("./routes/manuscripts");

const app  = express();
const PORT = process.env.PORT || 3000;

/* ---------- middleware ---------- */
app.use(cors({ origin: process.env.CLIENT_ORIGIN || "*" }));   // allow the React
client
app.use(express.json());

app.use((req, res, next) => {
  const t0 = Date.now();
  res.on("finish", () =>
    console.log(`${req.method} ${req.originalUrl} -> ${res.statusCode} (${Date.now() -
t0} ms)`)
  );
  next();
});

/* ---------- routes ---------- */
app.get("/api/health", (req, res) =>
  res.json({ status: "ok", uptimeSeconds: Math.round(process.uptime()) })
);

app.use("/api/manuscripts", manuscriptRoutes);     // mount the resource router

/* ---------- 404 ---------- */
app.use((req, res) =>
  res.status(404).json({ error: `No route for ${req.method} ${req.originalUrl}` })
);

/* ---------- centralised error handler ---------- */
app.use((err, req, res, next) => {
  console.error("Unhandled error:", err);
  res.status(500).json({ error: "Internal Server Error", detail: err.message });
});

app.listen(PORT, () =>
  console.log(`LipiSutra REST API listening on http://localhost:${PORT}`)
);
```

File 6: how the React client would consume it (replacing the direct API call)

```
// src/services/api.js  -  the client no longer holds any secret key
const BASE = import.meta.env.VITE_API_BASE || "http://localhost:3000/api";

export async function listManuscripts({ page = 1, script = "" } = {}) {
  const params = new URLSearchParams({ page, limit: 10 });
  if (script) params.set("script", script);

  const res = await fetch(`${BASE}/manuscripts?${params}`);
  if (!res.ok) throw new Error(`List failed: HTTP ${res.status}`);
  return res.json();
}

export async function createManuscript(record) {
  const res = await fetch(`${BASE}/manuscripts`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(record)
  });

  const data = await res.json();
  if (!res.ok) throw new Error(data.errors?.join("; ") || data.error);
```

```
  return data;
}

export async function deleteManuscript(id) {
  const res = await fetch(`${BASE}/manuscripts/${id}`, { method: "DELETE" });
  if (res.status === 204) return true;
  const data = await res.json();
  throw new Error(data.error);
}
```

Commands used to build and test

```
# set up
mkdir lipisutra-api && cd lipisutra-api
npm init -y
npm install express mysql2 cors dotenv

# create the schema (reuse lipisutra.sql from Assignment 13)
mysql -u root -p < lipisutra.sql

# run
node server.js

# ---- test every endpoint ----
curl http://localhost:3000/api/health

curl "http://localhost:3000/api/manuscripts?page=1&limit=2&script=Modi"

curl http://localhost:3000/api/manuscripts/1

curl -X POST http://localhost:3000/api/manuscripts \
     -H "Content-Type: application/json" \
     -d '{"accession":"LS-0104","title":"Sharada
Folio","script":"Sharada","era_year":1104}'

curl -X PATCH http://localhost:3000/api/manuscripts/4 \
     -H "Content-Type: application/json" \
     -d '{"status":"verified","confidence":88.5}'

curl -i -X DELETE http://localhost:3000/api/manuscripts/4
```

Output:

- Startup prints "MySQL pool ready." followed by "LipiSutra REST API listening on http://localhost:3000".
- GET /api/manuscripts?page=1&limit=2 returns HTTP 200 with {"page":1,"limit":2,"total":3,"pages":2,"data":[...]}, so the client can render pagination controls.
- GET /api/manuscripts?script=Modi returns only the Modi record; the filter is applied in SQL, not in JavaScript.
- GET /api/manuscripts/1 returns HTTP 200 and the record; GET /api/manuscripts/999 returns HTTP 404 with {"error":"No manuscript with id 999"}.
- POST with a valid body returns HTTP 201, a Location header of /api/manuscripts/4 and the created record.
- POST with {"accession":"LS-14","title":"abc"} returns HTTP 400 and the full error list, so the client can show every problem at once.
- POSTing LS-0104 a second time returns HTTP 409 with "accession LS-0104 already exists" - the duplicate-key error translated into the correct REST status.

- PATCH with {"status":"verified"} returns HTTP 200 and the updated record, leaving all other columns untouched.
- DELETE /api/manuscripts/4 returns HTTP 204 with an empty body; repeating it returns HTTP 404.
- Requesting an unknown path returns HTTP 404 as JSON rather than an HTML error page, which keeps the API uniform.
- The React client at http://localhost:5173 calls the API successfully because the cors middleware sends Access-Control-Allow-Origin for that exact origin.

Conclusion:

In this assignment, we developed a REST API using Node.js, Express and MySQL. We designed resource-oriented endpoints, mapped CRUD onto GET, POST, PUT, PATCH and DELETE, and returned the correct status code for every outcome including 201 with a Location header, 204 for a delete, 400 for validation failures, 404 for a missing resource and 409 for a duplicate key. We connected to the database through a mysql2 connection pool using parameterised queries, added pagination and filtering, enabled CORS for the React client and kept the credentials in environment variables. Finally, we showed how the LipiSutra client would consume this API instead of calling third-party services directly, which removes the bundled API key from the browser.

References:

10. Node.js Documentation, https://nodejs.org/docs/latest/api/
11. Express 5.x API Reference, https://expressjs.com/en/5x/api.html
12. MDN Web Docs - Express/Node introduction, https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction
13. mysql2 - Promise API documentation, https://sidorares.github.io/node-mysql2/docs
14. MDN Web Docs - HTTP response status codes, https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status
15. MDN Web Docs - Cross-Origin Resource Sharing (CORS), https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS